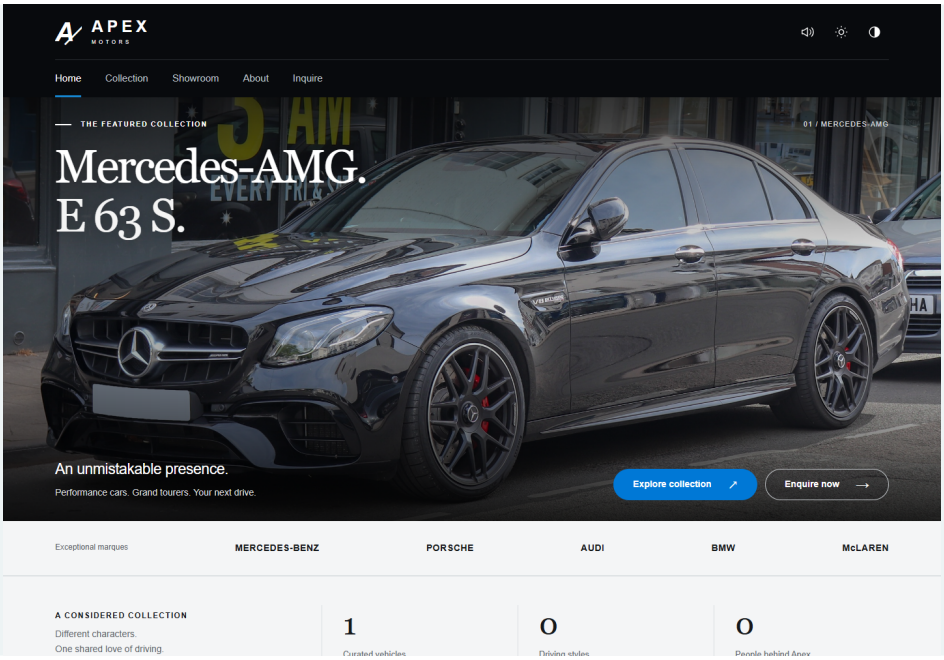
CONTINUING THE APEX MOTORS EXPERIENCE

Assignment 6. jQuery, in motion.

A polished dealership experience with live search, responsive feedback and purposeful interactions.



Team: Apex Motors **Group:** SE-2507
Members: Arsen, Kuttibay, Farabi, Muhammad

Objective. Continue the existing dealership project and use jQuery to simplify DOM manipulation, event handling and animation. This report covers every numbered task from 0 to 9 and the deployment requirement.

Scope. Five linked HTML pages, the original 17-car collection, local photographs, shared JavaScript and responsive CSS. The inquiry is a browser-local demonstration and sends no personal data.

Website: https://vertex-blip.github.io/Car-dealership-website/assignment6-jquery/

Prepared for the 5 November 2026 submission. Browser evidence is generated from the delivered source, with precise test records retained in screenshots/.

TASK 00

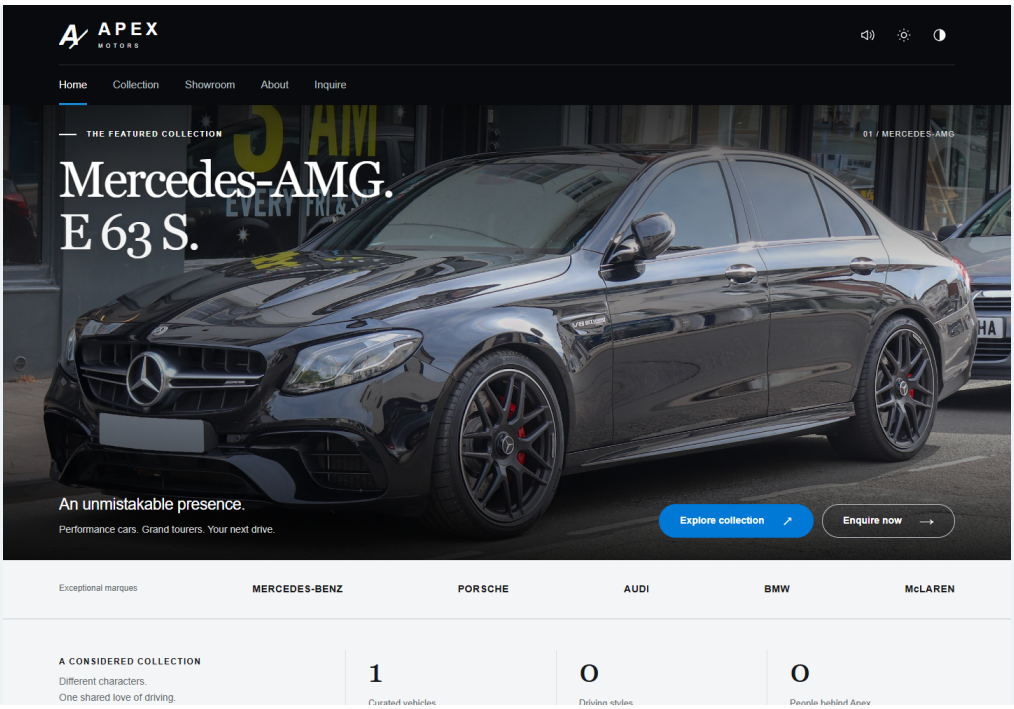
Ready for jQuery.

Load the exact library required by the brief, then initialize interactions after the DOM is ready.

1. Add the jQuery 4.0.0 CDN script to the head of all five HTML pages. Keep a local fallback for the downloadable ZIP.

2. Load external script.js after the library. Place shared initialization inside $(document).ready().

3. Print the exact message "jQuery is ready!". The browser checks confirm this message on every page and verify version 4.0.0.



Home page running with jQuery; the same setup is shared by all five pages.

index.html | lines 14-16

```
14  <script src="https://code.jquery.com/jquery-4.0.0.min.js"></script>
15  <script src="js/jquery-fallback.js?v=bc0431b80018"></script>
16  <link rel="stylesheet" href="styles.css?v=126232f49779" />
```

script.js | lines 2-5

```
2  $(document).ready(function () {
3    "use strict";
4    console.log("jQuery is ready!");
5    const Apex = (window.Apex = window.Apex || {});
```

Literal source-code screenshot from the delivered project; line numbers identify the exact excerpt.

TASK 01

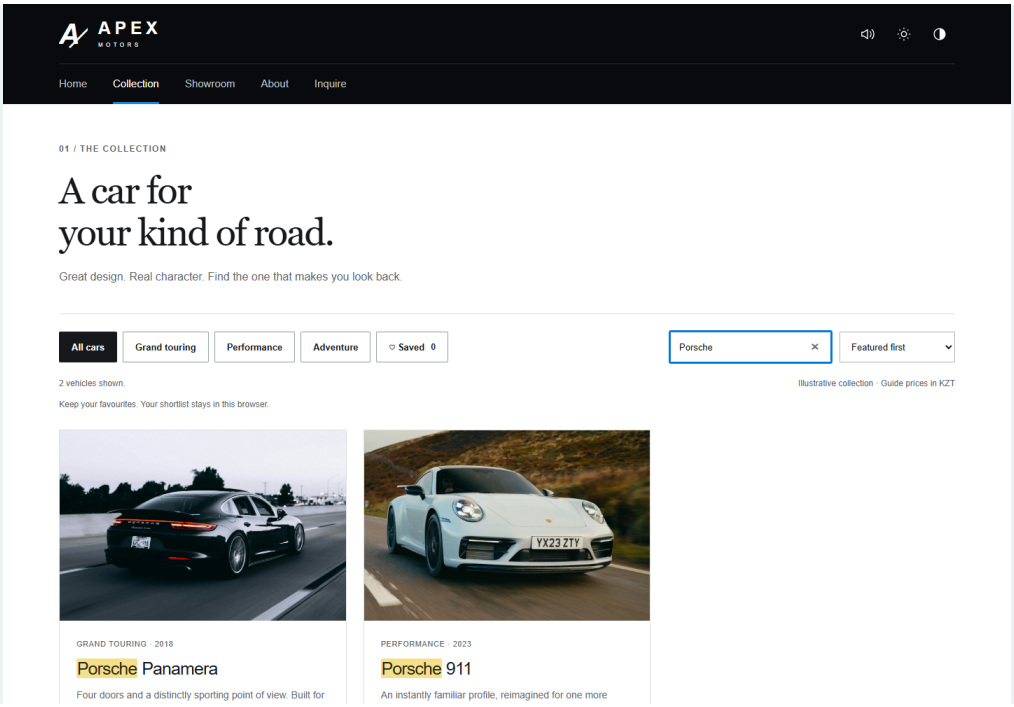
Search as you type.

Filter the real 17-vehicle collection immediately, without reloading or submitting a search form.

1. Listen to the search input with jQuery .on("input") so typing, pasting and clearing all update the collection.

2. Combine the text query with the active category and saved-car state, then render the matching vehicle objects.

3. Update the result count, highlight matching titles and show a useful empty state. Clearing restores all applicable cards.



Typing Porsche displays the two matching vehicles and highlighted names.

js/inventory.js | lines 166-171

```
166  const selectedVehicles = $(vehicles)
167    .filter(function (_, vehicle) {
168      return matchesCategory(vehicle) && matchesSearch(vehicle) &&
169        (!savedOnly || savedIds.includes(vehicle.id));
170    }).get();
171  switch (sortSelect?.value) {
```

js/inventory.js | lines 221-225

```
221  $(searchInput).on("input", function () {
222    render(false);
223    showSuggestions();
224    $("#clearVehicleSearch").prop("hidden", !this.value);
225  });
```

Literal source-code screenshot from the delivered project; line numbers identify the exact excerpt.

TASK 02

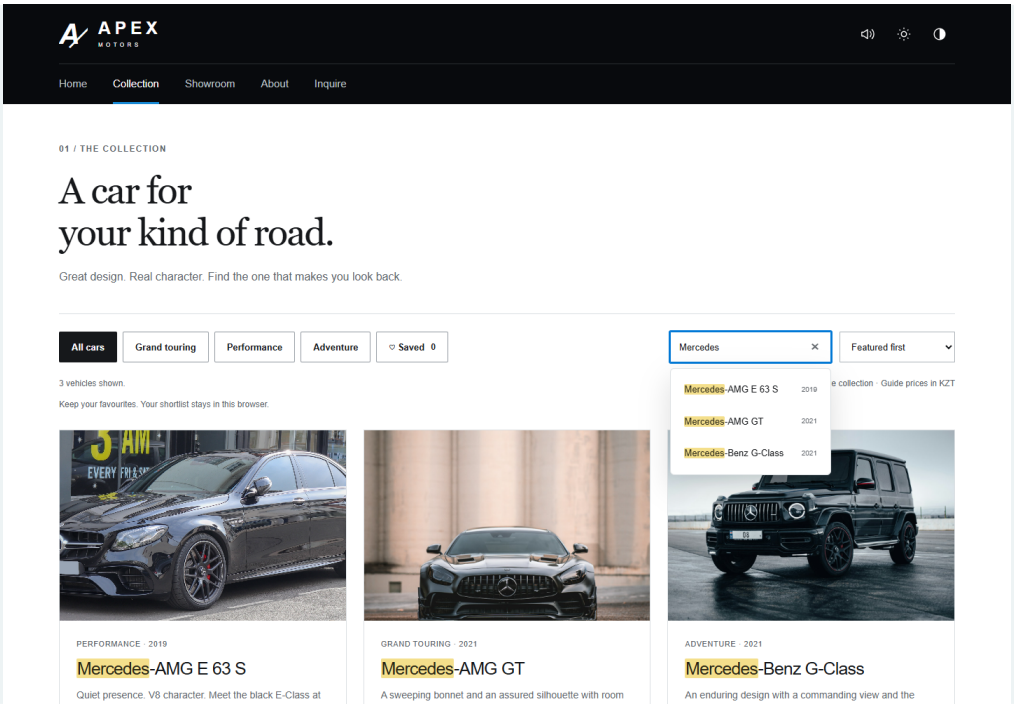
Find the right name.

Offer useful autocomplete suggestions while keeping the search usable with a keyboard and mouse.

1. Build a maximum of six suggestions from the same vehicle data and active filters. Give each option a stable ID.

2. Expose combobox, listbox and selected-option ARIA state. Arrow keys select; Enter accepts; Escape or Tab closes.

3. A mouse click chooses the same suggestion. The selected full name fills the field and filters the cards immediately.



Three Mercedes suggestions appear beneath the active search field.

js/inventory.js | lines 237-246

```
237  function showSuggestions() {
238    const query = searchInput.value.trim().toLocaleLowerCase();
239    $suggestions.empty();
240    activeSuggestion = -1;
241    $(searchInput).removeAttr("aria-activedescendant");
242    suggestions = query ? $(vehicles).filter(function (_, vehicle) {
243      return matchesCategory(vehicle) && matchesSearch(vehicle) &&
244        (!savedOnly || savedIds.includes(vehicle.id));
245    }).get().slice(0, 6) : [];
246    suggestions.forEach(function (vehicle, index) {
```

js/inventory.js | lines 261-267

```
261  function selectSuggestion(index) {
262    const selected = suggestions[index];
263    if (!selected) return;
264    $(searchInput).val(selected.name);
265    $("#clearVehicleSearch").prop("hidden", false);
266    render(false);
267    closeSuggestions();
```

Literal source-code screenshot from the delivered project; line numbers identify the exact excerpt.

TASK 03

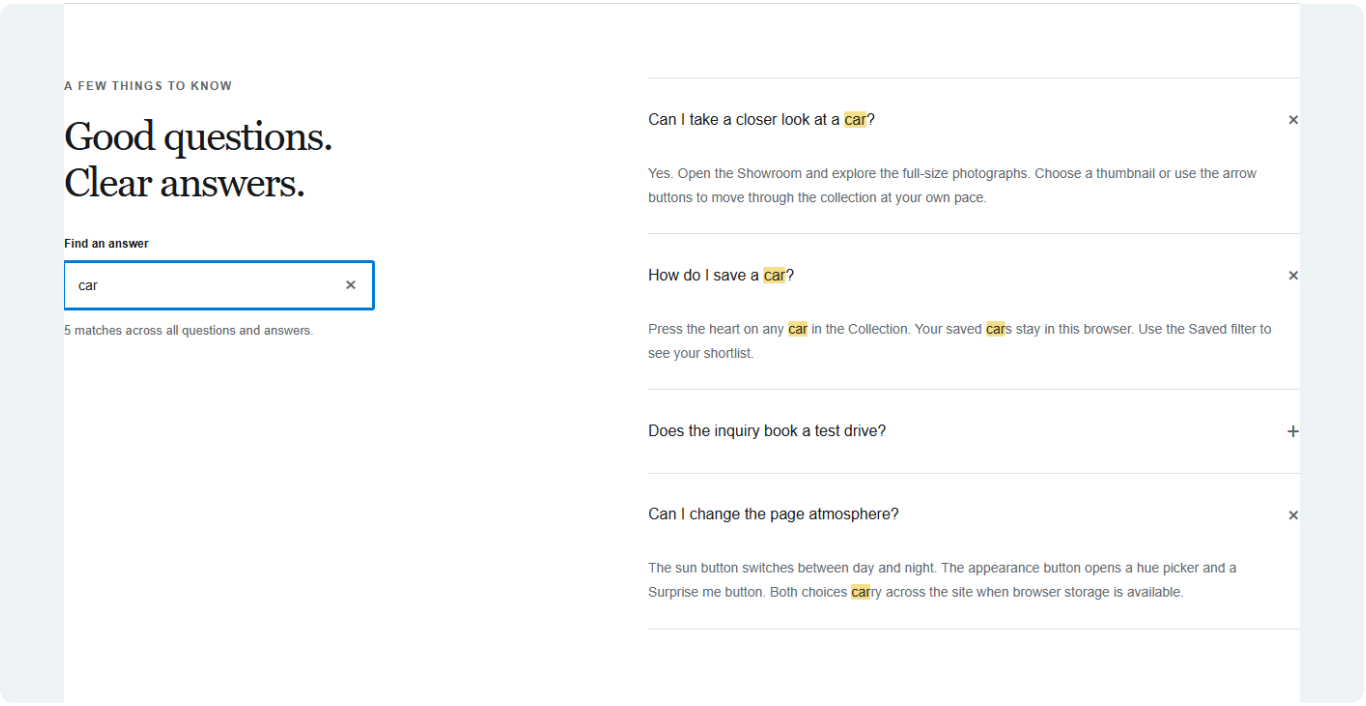
Make every match visible.

Highlight all literal keyword occurrences in FAQ questions and answers, while preserving the original markup.

1. Remove only previously generated mark elements, normalize text nodes, and escape regex metacharacters in the query.

2. Walk eligible text nodes and wrap each match in a mark element. Keep the whole question in one text wrapper so highlighted fragments remain inline beside the separate icon.

3. Open answers containing matches, announce the total and restore previous open states when the search is cleared.



The FAQ shows all occurrences of car, including matches inside expanded answers.

js/search.js | lines 14-21

```
14  const escaped = keyword.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
15  const pattern = new RegExp(escaped, "gi");
16  const walker = document.createTreeWalker(element, NodeFilter.SHOW_TEXT, {
17    acceptNode(node) {
18      return node.parentElement.closest("script, style, button, input, textarea, .visually-hidden")
19        ? NodeFilter.FILTER_REJECT : NodeFilter.FILTER_ACCEPT;
20    },
21  });
```

js/search.js | lines 33-38

```
33  const mark = document.createElement("mark");
34  mark.className = "search-highlight";
35  $(mark).text(match[0]);
36  fragment.append(mark);
37  start = match.index + match[0].length;
38  total += 1;
```

Literal source-code screenshot from the delivered project; line numbers identify the exact excerpt.

TASK 04

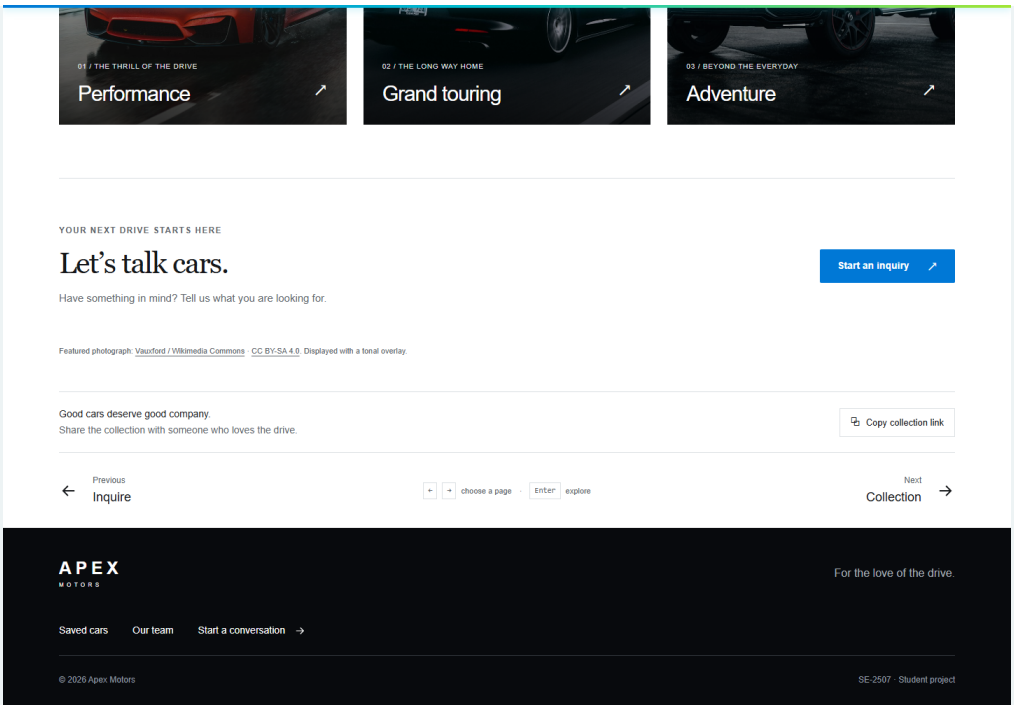
A sense of progress.

Show a colorful reading progress bar that responds to scrolling, resizing and changes in page height.

1. Insert a fixed gradient progress bar with an accessible 0-100 value on every page.

2. Calculate scroll position divided by available scroll distance. Clamp the percentage and handle a page with no scrollable distance.

3. Update width and aria-valuenow on scroll and resize; ResizeObserver also handles content height changes after filtering.



At the bottom of the page the colored progress bar is completely filled.

script.js | lines 12-21

```
12  function updateProgress() {
13    const distance = Math.max(0, document.documentElement.scrollHeight - window.innerHeight);
14    const percent = distance ? Math.min(100, Math.max(0, window.scrollY / distance * 100)) : 100;
15    $progress.attr("aria-valuenow", Math.round(percent)).find("span").css("width", percent + "%");
16  }
17  let frame;
18  $(window).on("scroll.apex resize.apex", function () {
19    if (!frame) frame = requestAnimationFrame(function () { frame = null; updateProgress();
    checkViewport(); });
20  });
21  if ("ResizeObserver" in window) new ResizeObserver(updateProgress).observe(document.body);
```

Literal source-code screenshot from the delivered project; line numbers identify the exact excerpt.

TASK 05

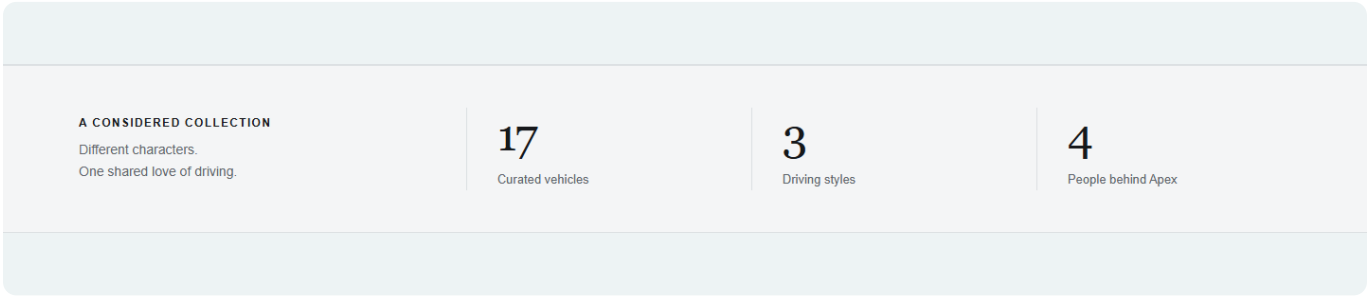
Numbers with meaning.

Animate real project statistics: 17 vehicles, three driving styles and four team members.

1. Keep each final value in data-count-to and display a starting value of zero.

2. Start a jQuery numeric animation when the statistics enter the viewport; update the displayed integer at each step.

3. Finish on the exact target and mark the counter as counted so it runs once. Reduced-motion preference resolves immediately.



The counters settle on factual collection and team totals.

script.js | lines 86-98

```
86  function animateCounter(element) {
87    const $number = $(element);
88    if ($number.attr("data-counted")) return;
89    $number.attr("data-counted", "true");
90    const target = Number($number.attr("data-count-to"));
91    $({ value: 0 }).animate({ value: target }, {
92      duration: motion.matches ? 0 : 1400, easing: "swing",
93      step: function (now) { $number.text(Math.floor(now).toLocaleString("en-US")); },
94      complete: function () { $number.text(target.toLocaleString("en-US")); }
95    });
96  }
97  const imageObserver = "IntersectionObserver" in window ? new IntersectionObserver(function (entries) {
98    entries.forEach(function (entry) { if (entry.isIntersecting) { loadImage(entry.target);
    imageObserver.unobserve(entry.target); } });
```

Literal source-code screenshot from the delivered project; line numbers identify the exact excerpt.

TASK 06

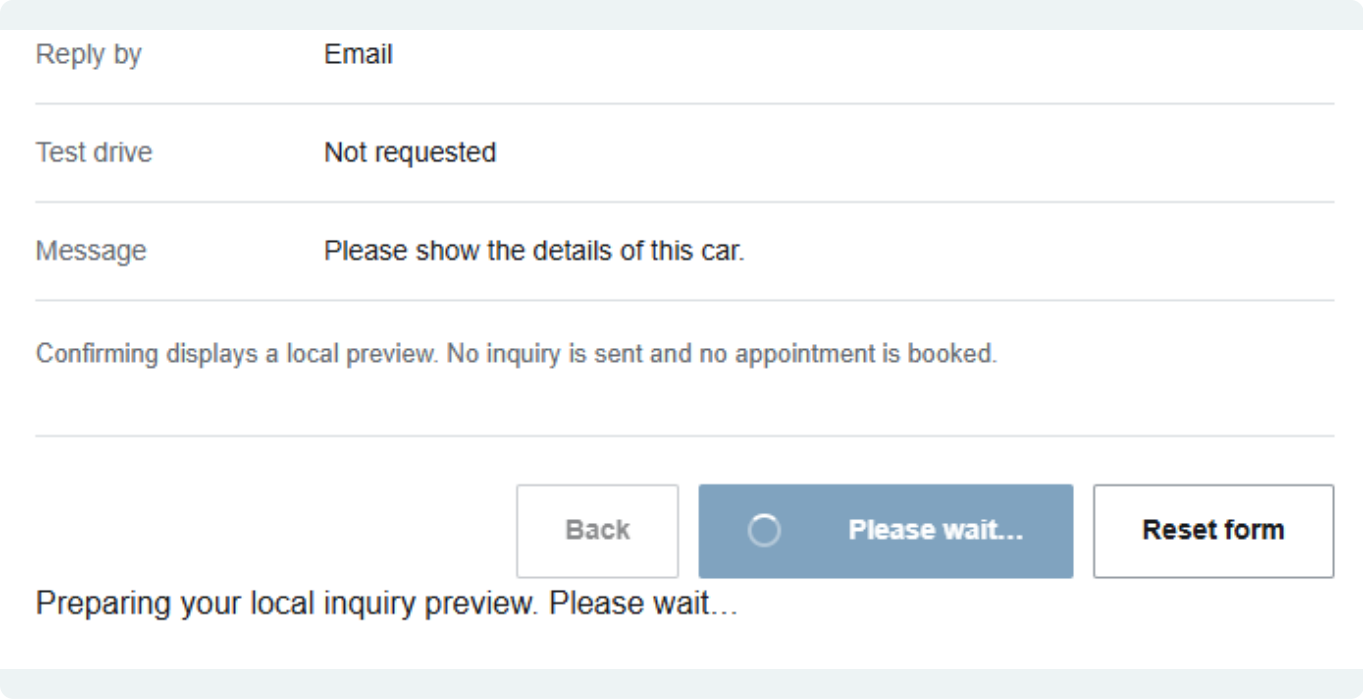
Submit with confidence.

Give clear feedback during a simulated form submission and prevent duplicate actions.

1. Validate the existing three-step inquiry. A valid submit starts a 1.3-second classroom simulation.

2. Show a spinner and Please wait, mark the form busy, disable submission and editing, and ignore repeated submits.

3. Restore the original button state and show the local preview after completion. Reset cancels the pending timer and clears the form.



The pending inquiry shows Please wait and a visible spinner; the submit button is disabled.

js/booking.js | lines 46-58

```
46  function startSubmit() {
47    pending = true;
48    lockedControls = $(form).find(":input").not("#resetInquiry").get().map(
49      (control) => ({ control, disabled: control.disabled }),
50    );
51    $(lockedControls.map(({ control }) => control)).prop("disabled", true);
52    $(submitButton)
53      .prop("disabled", true)
54      .html('<span class="button-spinner" aria-hidden="true"></span> Please wait…');
55    $(form).attr("aria-busy", "true");
56    $(status).text("Preparing your local inquiry preview. Please wait…");
57  }
58
```

js/booking.js | lines 305-305

```
305  if (pending || form.hidden) return;
```

Literal source-code screenshot from the delivered project; line numbers identify the exact excerpt.

TASK 07

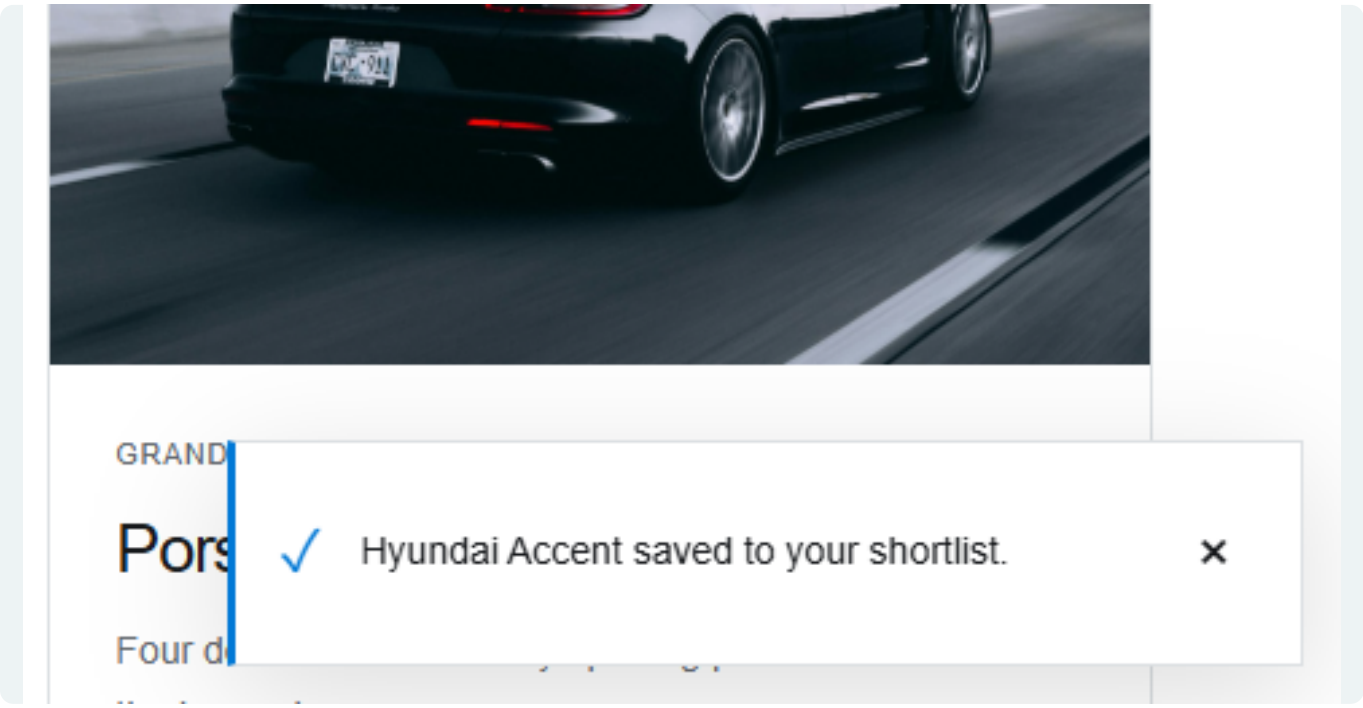
A small confirmation.

Confirm actions with temporary notifications that appear smoothly and disappear automatically.

1. Create toast content safely with jQuery and insert it into a polite live region.

2. Fade in the message after saving a vehicle, copying text or completing an inquiry. Keep no more than three notifications.

3. Fade out after 4.2 seconds or allow manual dismissal. Hover and focus pause the timer so the message can be read.



Saving a vehicle produces a visible, truthful confirmation.

script.js | lines 28-36

```
28  $toast.find("p").text(message);
29  if (kind === "error") $toast.addClass("toast-error").find(".toast-symbol").text("!");
30  if ($toasts.children().length >= 3) $toasts.children().first().stop(true, true).remove();
31  $toast.hide().appendTo($toasts).fadeIn(180);
32  let timer;
33  const dismiss = function () { clearTimeout(timer); $toast.stop(true).fadeOut(280, function () {
    $(this).remove(); }); };
34  const startTimer = function () { clearTimeout(timer); timer = setTimeout(dismiss, 4200); };
35  $toast.on("mouseenter focusin", function () { clearTimeout(timer); }).on("mouseleave focusout",
    startTimer);
36  $toast.find("button").on("click", dismiss);
```

Literal source-code screenshot from the delivered project; line numbers identify the exact excerpt.

TASK 08

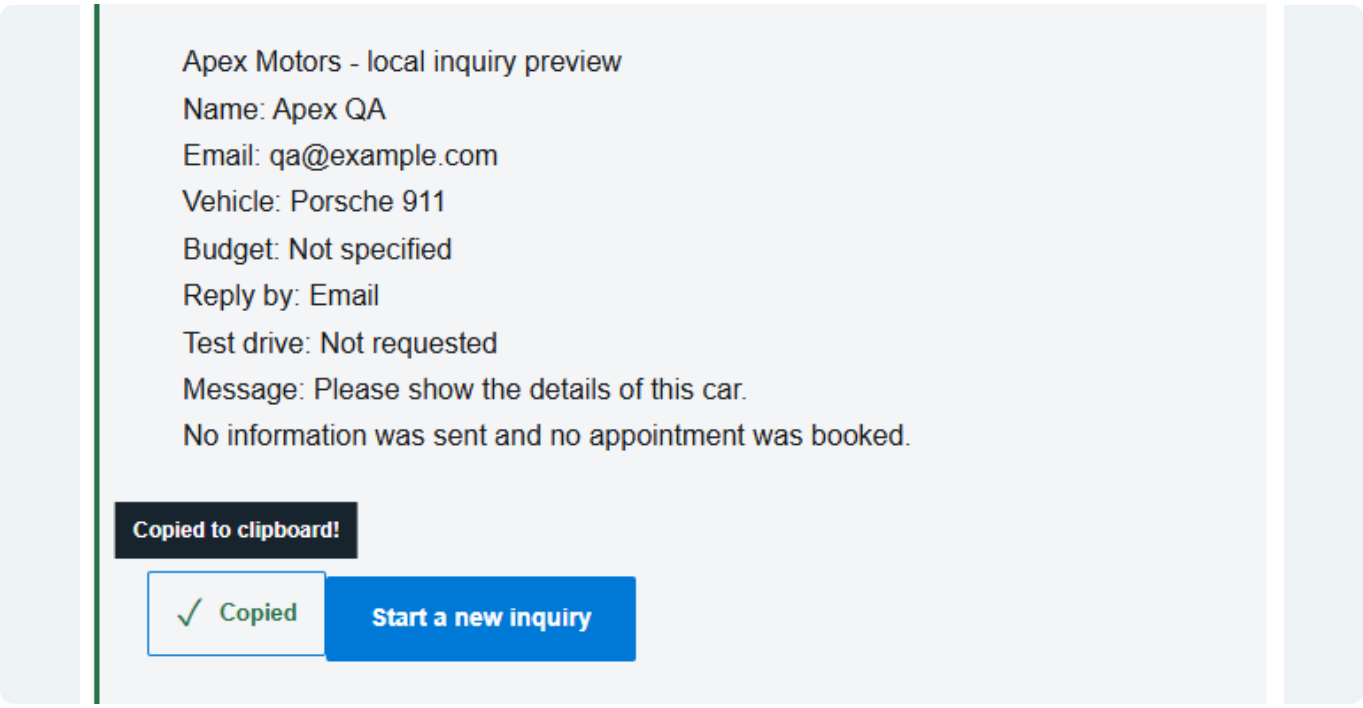
Copy it. Keep it.

Copy the actual visible snippet and confirm success with a checkmark and tooltip.

1. Read the text associated with the Copy button. Prefer navigator.clipboard.writeText in a secure context.

2. If unavailable, attempt a selected-text copy fallback. Report failure clearly if copying is denied.

3. Only after a successful write, replace the icon with a checkmark and show Copied to clipboard. Restore the button after a short delay.



The completed inquiry preview can be copied; checkmark and tooltip confirm success.

script.js | lines 59-67

```
59    await writeClipboard(value);
60    $button.find(".copy-icon").text("✓");
61    $button.find(".copy-label").text("Copied");
62    $('<span class="copy-tooltip" role="tooltip">Copied to clipboard!</span>').appendTo($button);
63    $button.addClass("is-copied");
64    Apex.toast("Copied to clipboard!");
65  } catch (_) {
66    Apex.toast("Copy is unavailable. Select the text and copy it manually.", "error");
67  } finally {
```

Literal source-code screenshot from the delivered project; line numbers identify the exact excerpt.

TASK 09

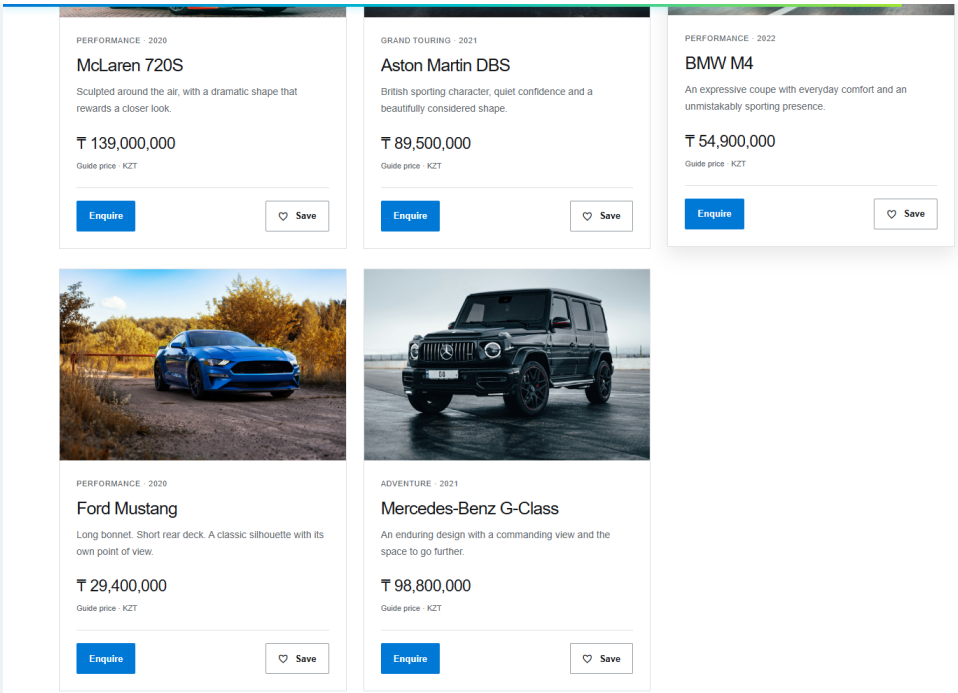
Load when it matters.

Defer offscreen photographs until they enter the viewport, reducing unnecessary initial image requests.

1. Keep the real image URL in data-src and use a local placeholder with fixed dimensions to avoid layout shifts.

2. Observe images entering the viewport, then use jQuery .attr("src", source) to request the real image. New filtered cards are observed too.

3. Unobserve loaded images and handle errors with the placeholder. A scroll/resize visibility check supports browsers without IntersectionObserver.



The last collection images load when scrolled into view; network checks confirm the request was deferred.

script.js | lines 78-86

```
78  function loadImage(image) {
79    const $image = $(image), source = $image.attr("data-src");
80    if (!source) return;
81    $image.removeAttr("data-src").addClass("is-loading");
82    $image.one("load", function () { $(this).removeClass("is-loading").addClass("is-loaded"); });
83    $image.one("error", function () { $(this).removeClass("is-loading").addClass("image-
    unavailable").attr("src", "assets/image-placeholder.svg"); });
84    $image.attr("src", source);
85  }
86  function animateCounter(element) {
```

script.js | lines 97-99

```
97  const imageObserver = "IntersectionObserver" in window ? new IntersectionObserver(function (entries) {
98    entries.forEach(function (entry) { if (entry.isIntersecting) { loadImage(entry.target);
    imageObserver.unobserve(entry.target); } });
99  }, { rootMargin: "0px", threshold: 0.01 }) : null;
```

Literal source-code screenshot from the delivered project; line numbers identify the exact excerpt.

PART 4 / PUBLICATION

From project to live site.

1. Preserve the original. Continue the existing shared repository in assignment6-jquery/; the Assignment 5 address remains available.

2. Verify locally. Run the browser suite against the static server. It checks all five pages, responsive widths, keyboard search, safe highlighting, notifications, clipboard, form lifecycle and actual image request deferral.

3. Commit and publish. Commit the Assignment 6 source and push the shared GitHub repository. GitHub Pages rebuilds the new folder.

4. Verify the deployed site. Repeat the browser suite against the live URL and retain the commit, workflow and live result in deployment.json and screenshots/test-results-live.json.

Website:
https://vertex-blip.github.io/Car-dealership-website/assignment6-jquery/
Repository:
https://github.com/Vertex-blip/Car-dealership-website

Publication status: verified live.
Application commit: 44205890704abcc8c082d433448533d521d70872
Successful GitHub Pages workflow

Verification evidence

Local HTTP: 122/122 checks passed; 0 authored errors.
Checked 2026-11-03T10:43:14.092Z; browser: chrome.

Direct-file: 121/121 checks passed; 0 authored errors.
Checked 2026-11-03T10:43:14.107Z; browser: chrome.

Live GitHub Pages: 122/122 checks passed; 0 authored errors.
Checked 2026-11-03T10:45:28.038Z; browser: chrome.

Focused FAQ layout regression: 35/35 checks passed; 0 authored errors.
Checked 2026-11-03T10:19:51.888Z; browser: chrome.

Live cached stylesheet regression: 11/11 checks passed; 0 authored errors.
Checked 2026-11-03T10:30:10.915Z; browser: chrome.

Submission files. Upload Assignment-6-Report.pdf and Assignment-6-Project.zip. The ZIP contains all pages, source scripts, styles, local assets, documentation and evidence. Each member submits separately in the LMS.

Evaluation: search 20%; engagement 20%; functionality 10%; defense 40%; report and deployment 10%. The implementation and report support the defense, which each member must perform personally.

REFLECTION / DEFENSE

What we learned.

One state, consistent results. Live search must combine with category, saved-car and sorting controls. Suggestions use the same data and filters, so the dropdown does not promise a car that the current view excludes.

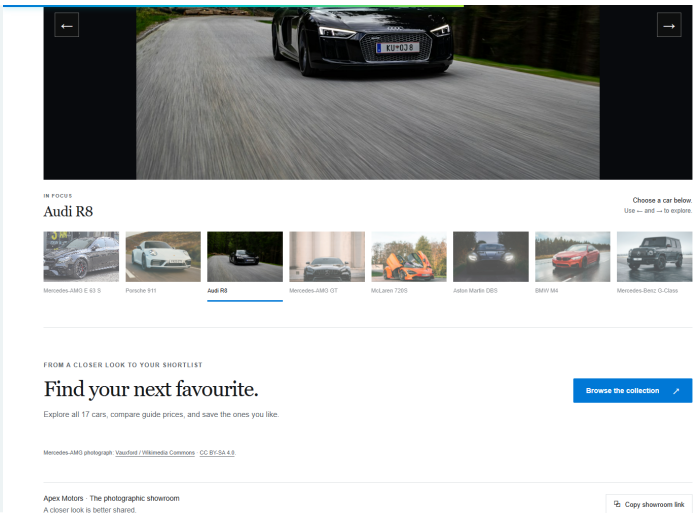
Treat input as text. Search terms may contain regex characters or HTML-like strings. Escaping the regex and constructing text nodes prevents broken searches and unintended markup while preserving links and existing page structure.

Make asynchronous work predictable. A disabled button alone does not cover programmatic or keyboard submits. A pending guard prevents duplicate work; restoring control states and cancelling the timer on reset prevents stale success messages.

Measure performance behavior. Native loading hints do not prove a request was deferred. The implementation holds the URL in data-src, and browser evidence checks that an offscreen image requests its real source after scrolling into view.

Design for different situations. Keyboard autocomplete, focus handling, live regions, reduced-motion behavior, a local jQuery fallback and storage-error feedback keep the experience usable beyond the default desktop path.

Keep deployed assets consistent. Content hashes in CSS and JavaScript URLs prevent new HTML from being combined with an old cached stylesheet. Run tools/version_assets.py after asset changes and before publication.



The photographic showroom and original dealership identity continue from Assignment 5.

Defense preparation. Demonstrate Tasks 0-9, explain .on(), .filter(), .text(), .attr(), .prop(), .animate() and .fadeOut(), then show the live URL. DEFENSE-GUIDE.md provides a Kazakh walkthrough and practice questions.

Limits. This static student site demonstrates local state and a simulated inquiry. It does not connect to a dealership backend, make real reservations or verify vehicle availability and prices.